Appointment Scheduler Technical Progress Report

Software Design 1 SDN150S | Group Project 12

Prepared for Management | 3 November 2026

Repository: https://github.com/ZuluBarbie/AppointmentScheduler

Group 12

Zilungile Cwathi

Phakamani Nicholas Dlamini

Sekete Manthata

Liyabona Ndevu

Okuhle Ntweni

Reneloe Wondeful Phakwago

Lisakhanya Songqwaba

1 Introduction

The current Appointment Scheduler accepts valid appointments and displays them in a table. This report records the Day 2 implementation and its test results. Editing, deletion, reminders and schedule views remain under development. The application addresses Project 12 of the Software Design 1 brief (Cape Peninsula University of Technology, 2026).

2 Sub questions

How should appointment data be represented? How should invalid dates, times and titles be rejected? How can the interface display a new appointment without changing existing records when validation fails?

3 Discussion

3.1 Software design approach

The design separates appointment data from the interface. Appointment stores a title, date, start time, end time and optional location. Its constructor checks the business rules. SchedulerFrame extends JFrame and owns the input fields, appointment list and table model. Main opens the frame on the Swing event thread through invokeLater (Oracle, n.d.a).

3.2 Engineering method and tools

Development uses small working versions: Day 1 provides an empty window and Day 2 adds entry and validation. GitHub stores dated source updates. VS Code supports editing, while the JDK compiler and JAR tool build each version independently. Automated checks exercise appointment rules separately from the interface. GUI observations are recorded in the test log.

3.3 Required Java concepts

Constructors initialise Appointment and SchedulerFrame. Private fields and public methods control access. Final fields preserve validated values. Conditional statements reject invalid input; loops add field labels and controls. A try/catch block displays validation errors. The interface currently uses a programmatically constructed JFrame. NetBeans visual form integration remains pending; a JFrame subclass alone does not establish use of its visual designer.

4 Design artefacts

4.1 Pseudocode

```
OPEN scheduler window on Swing event thread
CREATE empty appointment list and read only table
WHEN Add appointment is selected
    READ title, date, start time, end time and location
    TRY
        PARSE date and times using strict formats
        REJECT blank title or title longer than 60 characters
        REJECT missing date or times
        REJECT end time at or before start time
        CREATE validated appointment
        ADD appointment to list and table
        SHOW success message and appointment count
    ON invalid input
        SHOW validation message
        LEAVE existing list and table unchanged
ON window close
    CLOSE window
```

4.2 UML class diagram

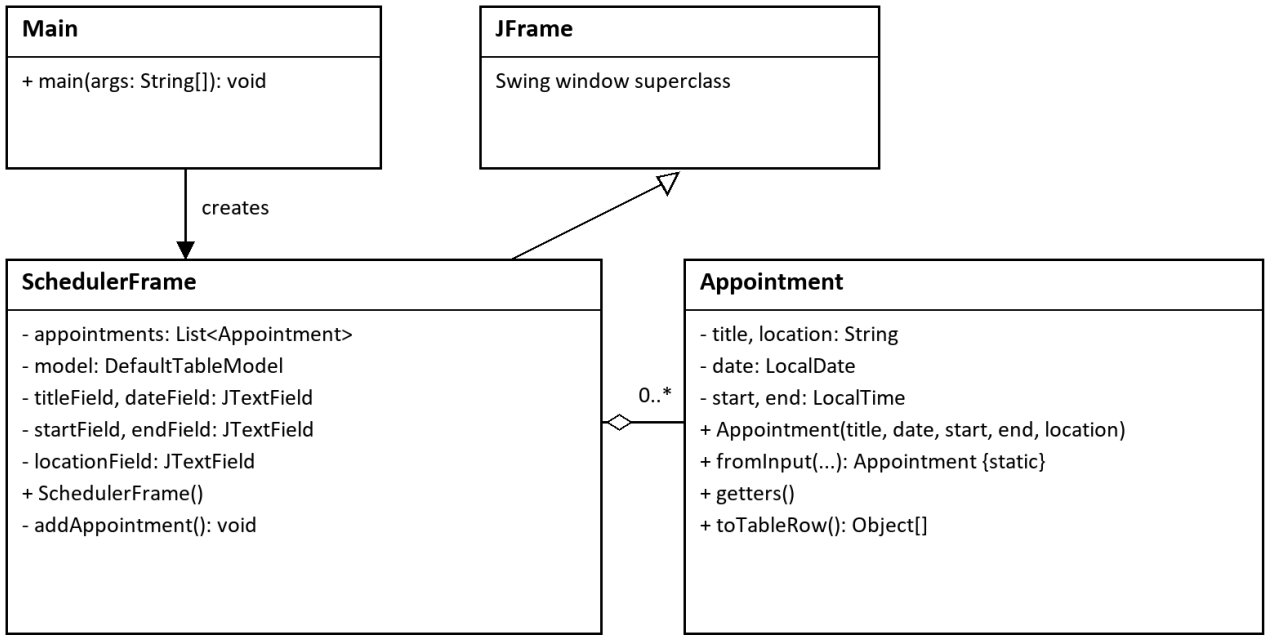


Figure 1. Day 2 classes and relationships. Field and method lists are abbreviated.

SchedulerFrame inherits from JFrame and holds zero or more appointments. Main creates the window. Appointment supplies validated values for each table row. Appendix A contains the Java implementation.

5 Results and analysis

The Day 2 build targets Java 21. All 15 appointment checks passed. They cover title handling and length boundaries, optional location, parsed values, table formatting, invalid and leap-year dates, incorrect formats, invalid times, equal or reversed times and a missing constructor date.

The Windows GUI check recorded an empty five-column table, rejection of a blank title, and successful addition of a valid appointment. Status feedback showed the new record and total. Invalid input is rejected before either the list or table is changed. Strict date resolution rejects calendar dates that do not exist (Oracle, n.d.b).

Appointments currently remain in memory until the window closes. Edit/delete actions, file storage, reminders and daily, weekly and monthly views are not implemented in Day 2. Recurrence, clash detection and calendar export are planned extensions. These limits prevent the current version from meeting the complete Project 12 requirements.

6 Conclusion

Day 2 provides appointment entry and validation through a working Java window. The tests support the implemented input rules. Completion by 6 November requires the remaining scheduler functions, NetBeans form work, final testing and an updated report and recording. The submission deadline is 8 November 2026.

References

Cape Peninsula University of Technology (2026) Software Design 1 SDN150S project brief. Department of Electrical, Electronic and Computer Engineering. Course handout.

Oracle (n.d.a) SwingUtilities. Java Platform Standard Edition 21 API Specification. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/SwingUtilities.html (Accessed: 3 November 2026).

Oracle (n.d.b) ResolverStyle. Java Platform Standard Edition 21 API Specification. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/format/ResolverStyle.html (Accessed: 3 November 2026).

Appendix A Java program

Day 2 source listing. Package: demo. Entry point: Main.main. Build and test commands are provided in the repository README.

Main class

```
package demo;


import javax.swing.SwingUtilities;
import javax.swing.UIManager;

/** Opens the Day 2 appointment window on Swing's event thread. */
public class Main {
   public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> {
         try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
         } catch (Exception ex) {
            System.err.println("Using default appearance: " + ex.getMessage());
         }
         new SchedulerFrame().setVisible(true);
      });
   }
}
```

Appointment class

```
package demo;


import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;

/** A validated appointment. Day 2 stores appointments only while the window is open. */
```

```
public final class Appointment {
   private static final DateTimeFormatter DATE_FORMAT = DateTimeFormatter
         .ofPattern("uuuu-MM-dd").withResolverStyle(ResolverStyle.STRICT);
   private static final DateTimeFormatter TIME_FORMAT = DateTimeFormatter
         .ofPattern("HH:mm").withResolverStyle(ResolverStyle.STRICT);

   private final String title;
   private final LocalDate date;
   private final LocalTime start;
   private final LocalTime end;
   private final String location;

   public Appointment(String title, LocalDate date, LocalTime start, LocalTime end,
         String location) {
      if (title == null || title.isBlank()) {
         throw new IllegalArgumentException("Enter an appointment title.");
      }
      if (title.trim().length() > 60) {
         throw new IllegalArgumentException("Use no more than 60 characters for the title.");
      }
      if (date == null || start == null || end == null) {
         throw new IllegalArgumentException("Date, start time and end time are required.");
      }
      if (!end.isAfter(start)) {
         throw new IllegalArgumentException("End time must be after start time on the same day.");
      }
      this.title = title.trim();
      this.date = date;
      this.start = start;
      this.end = end;
      this.location = location == null ? "" : location.trim();
   }

   /** Parse form values strictly so invalid dates cannot silently change day. */
```

```
    public static Appointment fromInput(String title, String date, String start,
            String end, String location) {
        LocalDate parsedDate;
        LocalTime parsedStart;
        LocalTime parsedEnd;
        try {
            parsedDate = LocalDate.parse(clean(date), DATE_FORMAT);
        } catch (DateTimeParseException ex) {
            throw new IllegalArgumentException("Enter a valid date as YYYY-MM-DD.");
        }
        try {
            parsedStart = LocalTime.parse(clean(start), TIME_FORMAT);
            parsedEnd = LocalTime.parse(clean(end), TIME_FORMAT);
        } catch (DateTimeParseException ex) {
            throw new IllegalArgumentException("Enter valid times as HH:mm, for example 09:00.");
        }
        return new Appointment(title, parsedDate, parsedStart, parsedEnd, location);
    }

    private static String clean(String value) { return value == null ? "" : value.trim(); }

    public String getTitle() { return title; }
    public LocalDate getDate() { return date; }
    public LocalTime getStart() { return start; }
    public LocalTime getEnd() { return end; }
    public String getLocation() { return location; }

    public Object[] toTableRow() {
        return new Object[]{title, date.toString(), start.format(TIME_FORMAT),
            end.format(TIME_FORMAT), location};
    }
}
```

SchedulerFrame class

```
package demo;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.GridLayout;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.table.DefaultTableModel;

/** Day 2 interface: add validated appointments to an in-memory schedule. */
public class SchedulerFrame extends JFrame {
    private final List<Appointment> appointments = new ArrayList<>();
    private final JTextField titleField = new JTextField();
    private final JTextField dateField = new JTextField(LocalDate.now().toString());
    private final JTextField startField = new JTextField("09:00");
    private final JTextField endField = new JTextField("10:00");
    private final JTextField locationField = new JTextField();
    private final JLabel feedback = new JLabel("Enter the details, then choose Add appointment.");
    private final DefaultTableModel model = new DefaultTableModel(
            new String[]{"Title", "Date", "Start", "End", "Location"}, 0) {
        @Override
        public boolean isCellEditable(int row, int column) { return false; }
    };
```

```
public SchedulerFrame() {
    super("Appointment Scheduler - Day 2: Add Appointments");
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    JPanel content = new JPanel(new BorderLayout(12, 12));
    content.setBorder(BorderFactory.createEmptyBorder(18, 18, 18, 18));

    JPanel top = new JPanel(new BorderLayout(8, 10));
    JLabel heading = new JLabel("My schedule");
    heading.setFont(heading.getFont().deriveFont(20f));
    top.add(heading, BorderLayout.NORTH);
    JPanel fields = new JPanel(new GridLayout(2, 5, 10, 4));
    String[] labels = {"Title (required)", "Date (YYYY-MM-DD)", "Start (HH:mm)",
        "End (HH:mm)", "Location (optional)"};
    for (String label : labels) { fields.add(new JLabel(label)); }
    JTextField[] inputs = {titleField, dateField, startField, endField, locationField};
    for (int i = 0; i < inputs.length; i++) {
        inputs[i].getAccessibleContext().setAccessibleName(labels[i]);
        fields.add(inputs[i]);
    }
    top.add(fields, BorderLayout.CENTER);
    JPanel actions = new JPanel(new BorderLayout(12, 0));
    JButton addButton = new JButton("Add appointment");
    addButton.addActionListener(event -> addAppointment());
    actions.add(addButton, BorderLayout.WEST);
    feedback.getAccessibleContext().setAccessibleName("Appointment status");
    actions.add(feedback, BorderLayout.CENTER);
    top.add(actions, BorderLayout.SOUTH);
    content.add(top, BorderLayout.NORTH);

    JTable table = new JTable(model);
    table.getAccessibleContext().setAccessibleName("Appointments");
    table.setFillsViewportHeight(true);
    table.setRowHeight(28);
```

```
        content.add(new JScrollPane(table), BorderLayout.CENTER);
        content.add(new JLabel("Appointments are kept only until this window closes."),
                BorderLayout.SOUTH);
        setContentPane(content);
        getRootPane().setDefaultButton(addButton);
        setMinimumSize(new Dimension(960, 440));
        setSize(1100, 540);
        setLocationRelativeTo(null);
    }

    private void addAppointment() {
        try {
            Appointment appointment = Appointment.fromInput(titleField.getText(),
                    dateField.getText(), startField.getText(), endField.getText(),
                    locationField.getText());
            // Validation completes before either the collection or table changes.
            appointments.add(appointment);
            model.addRow(appointment.toTableRow());
            feedback.setForeground(new Color(0, 100, 30));
            feedback.setText("Added: " + appointment.getTitle() + ". Total: " + appointments.size());
            titleField.setText("");
            locationField.setText("");
            titleField.requestFocusInWindow();
        } catch (IllegalArgumentException ex) {
            feedback.setForeground(new Color(170, 20, 20));
            feedback.setText(ex.getMessage());
        }
    }
}
```